

Milestone 1.1: Least Squares

INFO 521 · Machine Learning Foundations · project milestone brief ·
One Problem, Three Lenses · Lens 1 of 3

How to read this

This is the assignment brief for one project milestone. The Satisfactory criteria below are authoritative: every one must be met, and it is all-or-nothing rather than partial credit. A Not-Yet returns with targeted feedback and one revise cycle. Do the work in the notebook distributed through Classroom 50; this PDF is the brief, not the workspace.

Course-schedule placement. Part 1 of the project: due Week 5, the midpoint. This milestone covers M1 to M2 and is due Week 3. Gated by Checkpoint quiz 1.1 (normal equations, Wk 2).

The scenario

You are a clinical data scientist. You have one cohort and one question: **how does a patient's systolic blood pressure (SBP) change with age?** Across this project you will answer it three ways, least squares (here), then maximum likelihood, then a full Bayesian treatment, using the *same single predictor* throughout. The model order is your only complexity knob, exactly as in the Module 1–2 lectures.

Ground rules. Implement every estimator from scratch in NumPy/SciPy. Do **not** use scikit-learn or any off-the-shelf solver. The `info521` library gives you data, plotting, and self-checks only, never a fitting routine.

From scratch rules out libraries, never your own prior work: code you built and certified in a homework unit (here, Unit 1's design matrix, fits, and cross-validation) may be copied in or imported, and reusing it is encouraged. The homework certified that your components work; this milestone grades what you build with them.

What “Satisfactory” means for 1.1

You reach Satisfactory when **all** of the following hold:

- ☐ You build an n -th order polynomial **design matrix** $\mathbf{X}$ from the age predictor in NumPy.
- ☐ You implement OLS via the **normal equations** $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ and it matches the shipped self-check fixture within tolerance.
- ☐ You implement **K-fold cross-validation from scratch** and report training vs. validation error against model order.
- ☐ You add L_2 **regularization** and demonstrate coefficient shrinkage.
- ☐ **Play artifact** (below) is complete and interpreted.

- ☐ Checkpoint quiz 1.1 (normal equations) passed.
-

Setup

```
import numpy as np
from info521 import data, plotting, checks

plotting.use_course_style()
ds = data.load_clinical()
age = data.primary_predictor(ds)
y = ds["y"]
print(f"{len(y)} patients · predictor = {ds['primary']} · target = {ds['target']}")
```

1 · Design matrix

Write a function that maps the 1-D age vector to an order- p polynomial design matrix with columns $[1, x, x^2, \dots, x^p]$.

```
def design_matrix(x, order):
    # TODO: return an (n, order+1) ndarray of polynomial features.
    raise NotImplementedError
```

2 · Ordinary least squares: verify against the fixture first

Implement the normal equations, then confirm correctness on the fixed self-check case **before** trusting it on the real data.

```
def ols_fit(X, y):
    # TODO: solve the normal equations. Prefer np.linalg.solve over forming an
    # explicit inverse.
    raise NotImplementedError

# Self-check (ships expected values, not the algorithm):
fx = checks.LINEAR_FIXTURE
# checks.expect_close(ols_fit(fx.X, fx.y), fx.w_ols, "OLS weights")
```

Now fit age $\rightarrow$ SBP and plot the fit over a scatter of the data (age on the x-axis, systolic BP on the y-axis).

```
# TODO: choose an order, fit, predict on a grid, and plot with
# plotting.scatter(...) and plotting.fit_line(...).
```

3 · Cross-validation from scratch

```
def kfold_indices(n, k, seed=521):
    # TODO: return a list of (train_idx, val_idx) for K folds. Set the seed.
    # Your Unit 1 run_K_fold_cv satisfies "from scratch": bring it in and
    # adapt it rather than rewriting.
    raise NotImplementedError
```

```
# TODO: sweep polynomial order, compute mean train/val error per order,  
# then plotting.cv_curve(orders, train_err, val_err).
```

4 · L2 regularization

Add a ridge penalty. Recall the regularized solution $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + N\lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$.

```
def ridge_fit(X, y, lam):  
    # TODO. Self-check: checks.expect_close(ridge_fit(fx.X, fx.y, fx.lam_ridge),  
    #                                     fx.w_ridge, "ridge weights")  
    raise NotImplementedError
```

Play artifact: the bias–variance sweep

Fit the model across the **grid** of polynomial orders $\{1, 3, 9\}$ and regularization strengths $\lambda \in \{10^{-3}, 10^{-1}, 10^1\}$ (nine combinations). For each, plot the fit and record validation error.

Then **write 150–250 words** answering: which combinations underfit, which overfit, and what the regularization strength does to the coefficients and to the fitted curve. Tie your reading explicitly to the bias–variance tradeoff. Be honest about what you see: on this real, only weakly-nonlinear age → SBP signal the gain from higher polynomial order is small, and the dramatic textbook underfit/overfit contrast is itself somewhat synthetic, on real data the simplest adequate model often wins.

```
# TODO: produce the 3x3 grid of fits and the short written interpretation.
```

i Gate

This milestone is not Satisfactory until **Checkpoint quiz 1.1 (derive the normal equations)** is passed. The checkpoint is an open-book D2L quiz, due the week before this milestone; the auto-graded portion resolves on submission and written responses allow one revision.

Looking ahead

In 1.2 you will re-derive this exact fit as a **maximum likelihood** estimate under a Gaussian noise model, and show numerically that it lands on the same weights you found here.